`WebClient` object—in this example we'll use `WebClient.Builder`. Refer to the official documentation to explore the other options (https://spring.io/projects/spring-framework).

In the `com.polarbookshop.orderservice.config` package, create a `ClientConfig` class to configure a `WebClient` bean with the base URL provided by `ClientProperties`.

Listing 8.20 Configuring a `WebClient` bean to call Catalog Service

```
package com.polarbookshop.orderservice.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.reactive.function.client.WebClient;

@Configuration
public class ClientConfig {

  @Bean
  WebClient webClient(
    ClientProperties clientProperties,
    WebClient.Builder webClientBuilder                          ❶
  ) {
    return webClientBuilder                                     ❷
      .baseUrl(clientProperties.catalogServiceUri().toString())
      .build();
  }
}
```

❶ An object auto-configured by Spring Boot to build WebClient beans

❷ Configures the WebClient base URL to the Catalog Service URL defined as a custom property

WARNING If you use IntelliJ IDEA, you might get a warning that `WebClient .Builder` cannot be autowired. Don't worry. It's a false positive. You can get rid of the warning by annotating the field with `@SuppressWarnings("SpringJavaInjectionPointsAutowiringInspection")`.

Next, create a `BookClient` class in the `com.polarbookshop.orderservice.book` package. That is